OneCare Clinic Management System
System Design Specification

By

Thushajini Yogalingam
(SE/2021/019)
Vipusa Sriharan
(SE/2021/043)
Ramachandran Sweeskaran
(SE/2021/046)
Kenuja Sivakumar
(SE/2021/050)

A report submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science Honors in Software Engineering (B. Sc.SE)

Software Engineering Teaching Unit
Faculty of Science
University of Kelaniya
Sri Lanka
2026

Version History

Version	Date	Description	Author
1.0	2026-04-04	Initial Draft	Team
1.1	2026-04-10	Architecture Design, Deployment Design	Vipusa
1.2	2026-04-12	Security Design	Kenuja
1.3	2026-04-17	Database Design	Thushajini
1.4	2026-04-18	Class Diagram, Sequence Diagram	Sweeskaran
1.5	2026-04-19	UI/UX Design	Vipusa, Kenuja

Table of Contents

CHAPTER 1: ARCHITECTURAL DESIGN 1

1.1 High-level architecture 1
1.2 Cloud-Native Architecture 3
1.3 Justification for architecture choice 4
1.4 Scalability and Maintainability Considerations 4

CHAPTER 2: DATABASE DESIGN 6

2.1 ER Diagram 6
2.2 Normalization Explanation 7

CHAPTER 3: CLASS DIAGRAM 10

3.1 Description of the Class Diagram 11
3.2 Relationships Overview 11

CHAPTER 4: SEQUENCE DIAGRAM 13

4.1 Sequence Diagram for Patient 14
4.2 Sequence Diagram for Doctor 15
4.3 Sequence Diagram for Pharmacy Staff 16
4.4 Sequence Diagram for Admin 17
4.5 Sequence Diagram for Super Admin 18

CHAPTER 5: UI/UX DESIGN 20

5.1 Wireframes and Screen Mock-ups 20
5.2 User Flow and Navigation Design 23
5.3 Usability Considerations 24
5.4 Accessibility Considerations (WCAG 2.1 Compliance) 25

CHAPTER 6: SECURITY DESIGN 27

6.1 Authentication Mechanism 27
6.2 Authorization Levels 31
6.3 Data Protection Approach 34
6.4 Input Validation and Sanitization Strategy 36
6.5 Encryption Standards for data at rest and in transit 38

CHAPTER 7: DEPLOYMENT DESIGN 41

7.1 Hosting Plan 41
7.2 Hardware Requirements 41
7.3 Network Diagram 42
7.4 Environment specification 43
7.5 CI/CD Pipeline 45

List of Figures

FIGURE 1-1 3-TIER ARCHITECTURE 1
FIGURE 1-2 CLOUD NATIVE ARCHITECTURE 3
FIGURE 2-1 ER DIAGRAM 6
FIGURE 3-1 CLASS DIAGRAM 10
FIGURE 4-1 SEQUENCE DIAGRAM FOR LOGIN 13
FIGURE 4-2 SEQUENCE DIAGRAM FOR BOOK APPOINTMENT 14
FIGURE 4-3 SEQUENCE DIAGRAM FOR CHAT WITH CHATBOT 14
FIGURE 4-4 SEQUENCE DIAGRAM FOR SEARCH PATIENT 15
FIGURE 4-5 SEQUENCE DIAGRAM FOR VIEW MEDICAL HISTORY 15
FIGURE 4-6 SEQUENCE DIAGRAM FOR MANAGE INVENTORY 16
FIGURE 4-7 SEQUENCE DIAGRAM FOR GENERATE INVOICE 17
FIGURE 4-8 SEQUENCE DIAGRAM FOR REGISTER PATIENT 17
FIGURE 4-9 SEQUENCE DIAGRAM FOR AUDIT LOGS VIEW 18
FIGURE 4-10 SEQUENCE DIAGRAM FOR MANAGE USERS 18
FIGURE 4-11 SEQUENCE DIAGRAM FOR CONFIGURE ROLES 19
FIGURE 5-1 LOGIN SCREEN WIREFRAME 20
FIGURE 5-2 PHARMACIST DASHBOARD 21
FIGURE 5-3 ADMIN'S DASHBOARD 21
FIGURE 5-4 SUPER ADMIN'S DASHBOARD 22
FIGURE 5-5 DOCTOR'S DASHBOARD 22
FIGURE 5-6 USER FLOW DIAGRAM 24
FIGURE 6-1 AUTHENTICATION FLOW DIAGRAM 29
FIGURE 6-2 SECURITY ARCHITECTURE 39
FIGURE 7-1 NETWORK DIAGRAM 42
FIGURE 7-2 ENVIRONMENT SPECIFICATION 43
FIGURE 7-3 CI/CD PIPELINE ARCHITECTURE 46

List of Tables

TABLE 2-1 BEFORE 1NF PATIENT TABLE 7
TABLE 2-2 AFTER 1NF PATIENT TABLE 7
TABLE 2-3 BEFORE 1NF PRESCRIPTION ITEM TABLE 7
TABLE 2-4 AFTER 1NF PRESCRIPTION ITEM TABLE 7
TABLE 2-5 BEFORE 2NF PRESCRIPTION ITEM TABLE 8
TABLE 6-1 PASSWORD POLICY 27
TABLE 6-2 ROLE DEFINITIONS 31
TABLE 6-3 PERMISSION MATRIX 32
TABLE 6-4 DATA PROTECTION APPROACH 34
TABLE 6-5 RATE LIMITING SCENARIOS 37
TABLE 6-6 ENCRYPTION STANDARDS 38
TABLE 6-7 SECURITY TESTING 39

CHAPTER 1: ARCHITECTURAL DESIGN

1.1 High-level architecture

A Three-Tier Layered Architecture is adopted by the OneCare Clinic Integrated Pharmacy Management System, consisting of the Presentation Layer (Frontend), Application Layer (Backend), and Data Layer (Database). Separation of concerns, scalability, security, and maintainability are ensured through this architectural approach.

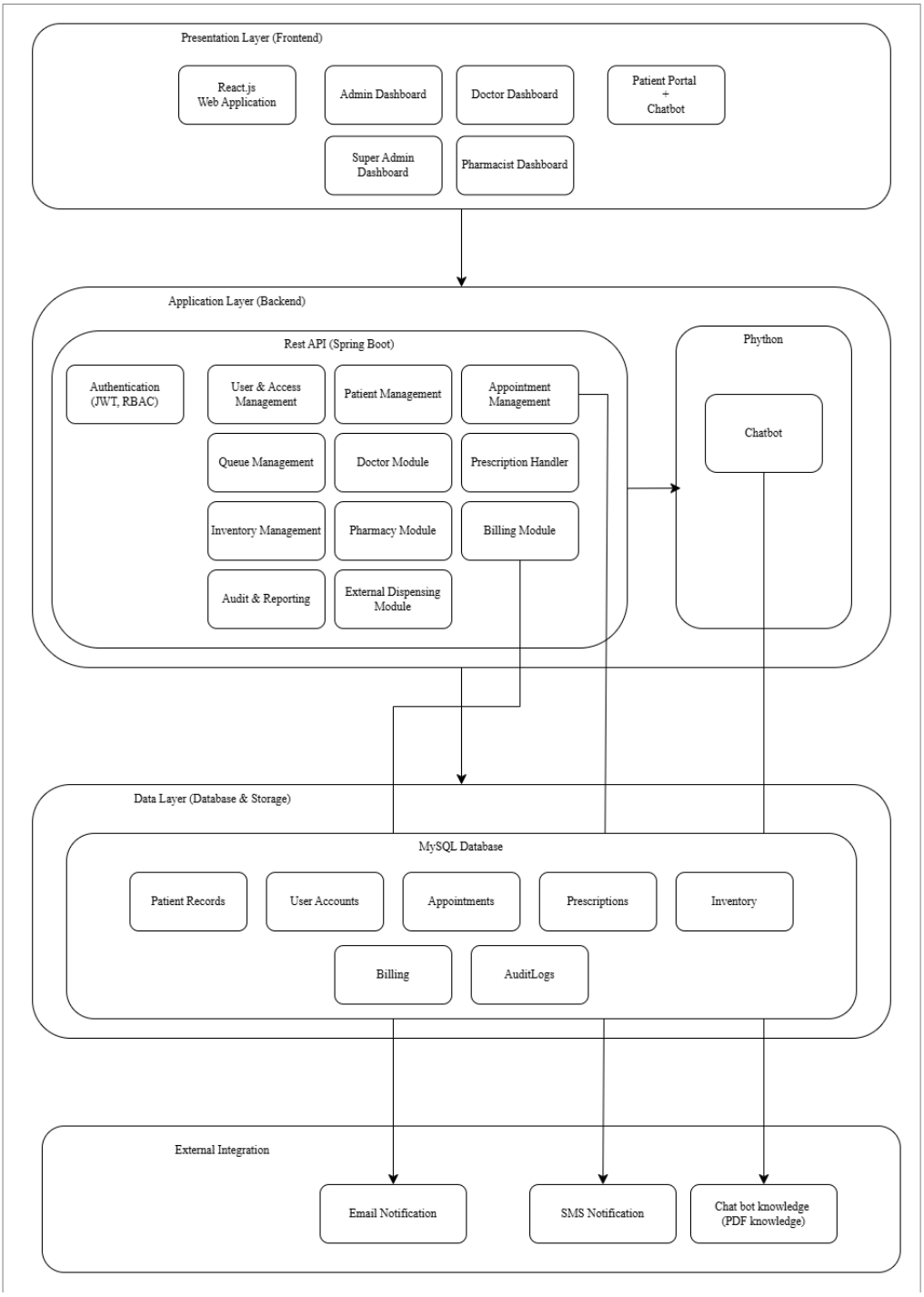


Figure 1-1 3-Tier Architecture

1.1.1 Architecture Overview

Presentation Layer (Frontend Components)

- React.js web application serving all interfaces
- Role-based dashboards for Admin, Doctor, Pharmacist, and Super Admin
- Patient self-service portal with chatbot integration
- All communicate via REST API

Application Layer (Backend Services)

The backend is organized around a Spring Boot REST API gateway with several key functional modules:

- **Security**: Authentication & Authorization using JWT and role-based access control (RBAC)
- **Core Business Modules**:
    - User & Access Management
    - Patient Management
    - Appointment Management
    - Queue Management
    - Doctor Module
    - Pharmacy Module
    - Inventory Management
    - Billing Module
    - Chatbot Service
    - Prescription Handler

Data Layer (MySQL Database)

- Central MySQL database storing all operational data
- Organized tables for patients, users, appointments, prescriptions, inventory, billing records, audit logs, and chatbot knowledge base

External Services (Third-Party Integrations)

- SMS Notification Service (for appointment reminders)
- Email Service (SMTP for confirmations and reports)
- Chatbot Knowledge Base Storage (PDF documents)

1.2 Cloud-Native Architecture

The OneCare system is designed as a cloud-native application with containerization, enabling scalability and deployment flexibility.

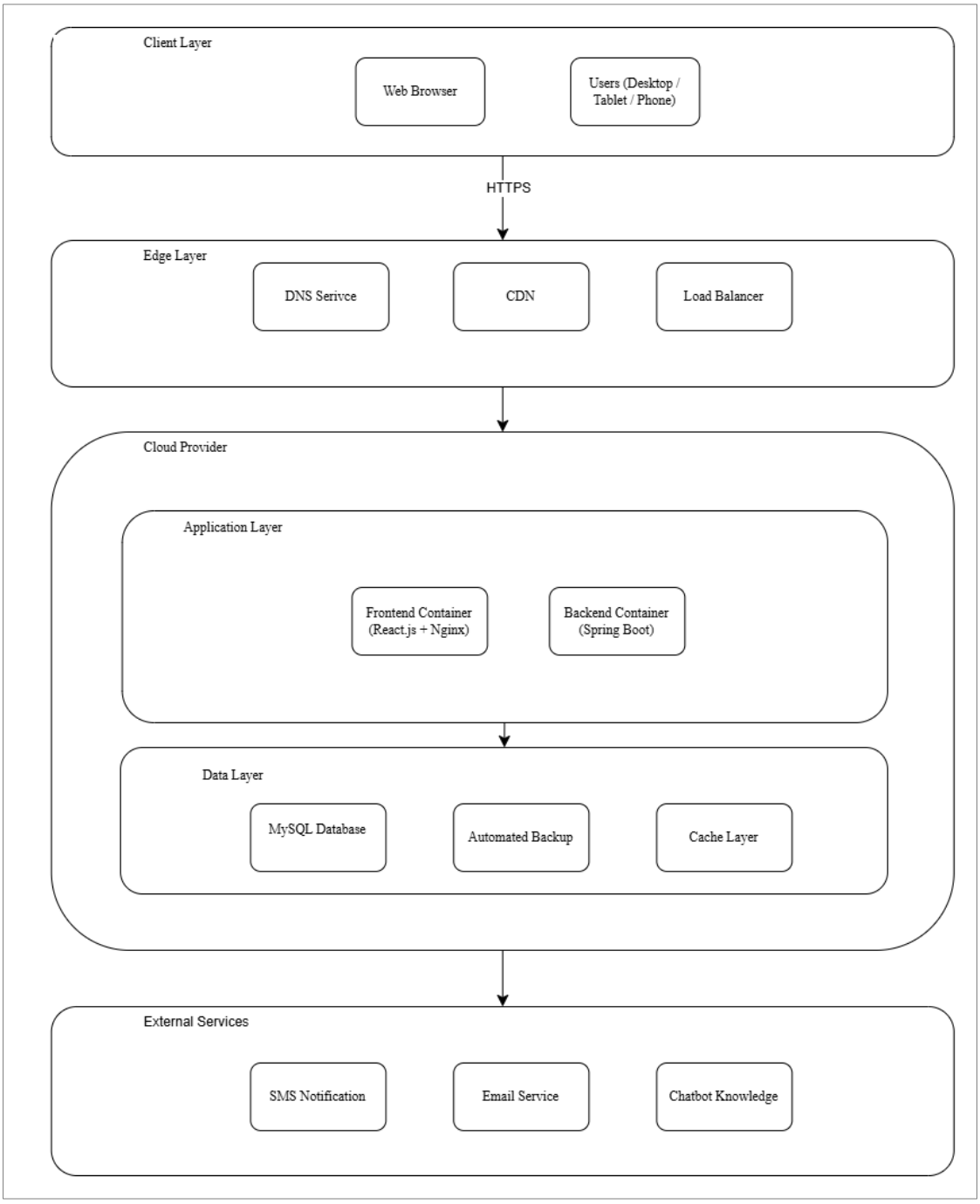


Figure 1-2 Cloud Native Architecture

1.3 Justification for architecture choice

1.3.1 Why Three-Tier Layered Architecture?

- **Separation of Concerns:** Clear division between presentation, business logic, and data access enables independent development and testing.
- **Scalability:** Each layer can be scaled independently. Frontend via CDN, backend via load balancing, database via optimization.
- **Security:** Clear enforcement points for authentication and authorization. Data access control isolated in data layer.
- **Maintainability:** Modular design allows feature additions without major refactoring. Clear API contracts between layers.
- **Cost-Effectiveness:** Leverages open-source technologies with no licensing fees. Cloud-native design reduces infrastructure costs.
- **Technology Alignment:** Matches proposed stack (React, Spring Boot, MySQL). All technologies are mature and widely supported.

1.3.2 Why NOT Microservices?

For a clinic system supporting ~25 patients/day, microservices architecture is not justified because:

- Operational complexity increases (service discovery, distributed transactions)
- Higher infrastructure costs and deployment complexity
- Overkill for the system scale and requirements
- Development team expertise better suited for layered monolithic backend

1.4 Scalability and Maintainability Considerations

1.4.1 Scalability Measures

- **Horizontal Scaling**: Stateless backend services can be scaled behind a load balancer
- **Performance Optimization**: API response ≤2 seconds, Dashboard load ≤3 seconds
- **Database Optimization**: Indexing on frequently queried fields, connection pooling
- **Caching Strategy**: Redis for frequently accessed data (optional)
- **Offline Capability**: Service Workers and Indexed DB for 5-minute offline operations

1.4.2 Maintainability Measures

- **Modular Architecture**: Separated modules per feature with clear responsibilities
- **Code Quality Standards**: ≥70% unit test coverage, SonarQube analysis
- **API Documentation**: Swagger/OpenAPI auto generated from annotations
- **Logging & Monitoring**: Centralized logging with 90-day retention
- **Version Control**: Git-based with CI/CD pipeline

CHAPTER 2: DATABASE DESIGN

2.1 ER Diagram

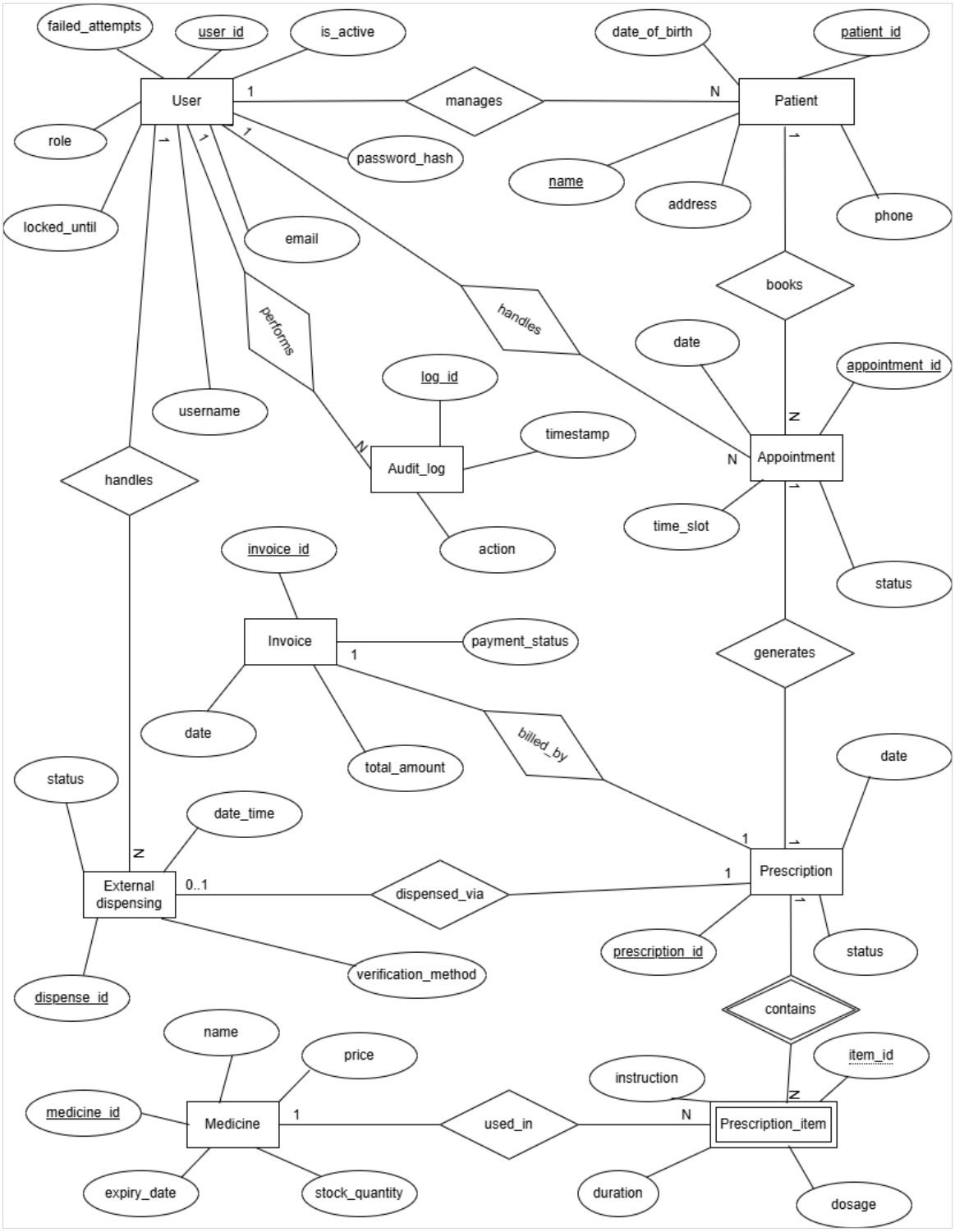


Figure 2-1 ER Diagram

2.2 Normalization Explanation

Normalization is a systematic process used in relational database design to minimize data redundancy and improve data integrity. It involves decomposing tables into smaller, well-structured relations based on defined rules known as Normal Forms (1NF, 2NF, and 3NF).

2.2.1 First Normal Form(1NF)

A relation is said to be in First Normal Form (1NF) when all attributes contain atomic (indivisible) values, repeating groups are eliminated, and each record is uniquely identifiable using a primary key.

A poorly designed Patient table in the proposed system is illustrated as follows:

Before 1NF

Table 2-1 Before 1NF Patient Table

patient_id	name	address	contact_number	date_of_birth
P001	Arun	jaffna	0756737217,0728605573	15.10.1997

After 1NF

Table 2-2 After 1NF Patient Table

patient_id	name	address	contact_number	date_of_birth
P001	Arun	jaffna	0728605573	15.10.1997
P001	Arun	jaffna	0756737217	15.10.1997

In the normalized structure, each contact number is stored as a separate record, ensuring atomicity and eliminating repeating groups.

In the proposed system, a poorly designed Prescription Item table is illustrated as follows:

Before 1NF

Table 2-3 Before 1NF Prescription Item Table

prescription_id	Medicines
1	Paracetamol 500mg, Amoxicillin 250mg
2	Ibuprofen 400mg

After 1NF

Table 2-4 After 1NF Prescription Item Table

item_id	prescription_id	medicine	dosage
1	1	Amoxicillin	250mg

2	1	Paracetamol	500mg
3	2	Ibuprofen	400mg

In the normalized structure, each medicine is stored as a separate record, ensuring atomicity and eliminating repeating groups.

2.2.2 Second Normal Form(2NF)

The relation is in Second Normal Form (2NF) if it is already in 1NF and all non-key attributes are fully functionally dependent on the entire primary key. This eliminates partial dependencies, which occur when a non-key attribute depends only on part of a composite key.

In the Prescription Item table, the following structure is observed:

Before 2NF

Table 2-5 Before 2NF Prescription Item Table

prescription_id	medicine_id	medicine_name	price	dosage	duration
1	M01	Paracetamol	5.00	500mg	5days
2	M01	Paracetamol	5.00	250mg	3days

After 2NF

Prescription Item table – item_id, prescription_id, medicine_id, dosage, duration, instruction
Medicine table – medicine_id, name, price, expiry_date, stock_quantity

In the unnormalized structure, attributes such as medicine_name and price depend only on medicine_id rather than the full composite key. This results in redundancy. By separating medicine details into a dedicated Medicine table, partial dependency is removed.

2.2.3 Third Normal Form(3NF)

A relation is in Third Normal Form (3NF) if it is already in 2NF and contains no transitive dependencies. This means that non-key attributes must depend only on the primary key and not on other non-key attributes.

Before 3NF

Prescription table - prescription_id, date, status, doctor_id, doctor_name, doctor_email

The attribute doctor_name depends on doctor_id rather than on prescription_id. This represents transitive dependency.

After 3NF

After normalization, the Prescription table stores only prescription_id, date, status, and doctor_id. Doctor-related attributes are maintained in the User table, where each user is assigned a role. The doctor_id acts as a foreign key referencing the corresponding user record.

In the unnormalized structure, the attributes doctor_name and doctor_email depend on doctor_id rather than directly on prescription_id, creating a transitive dependency. This dependency is removed by storing all user-related details in the User table and referencing them through doctor_id as a foreign key.

Thus, normalization ensures that the database structure is efficient, consistent, and free from update anomalies, supporting reliable data management within the system.

CHAPTER 3: CLASS DIAGRAM

This class diagram represents a Clinic Management System designed to handle patient care, appointments, prescriptions, and pharmacy operations. It models the structure of the system by showing the key classes, their attributes, methods, and relationships. The system involves multiple roles such as Patient, Doctor, Admin, Pharmacy Staff, and Super Admin, each responsible for different functionalities.

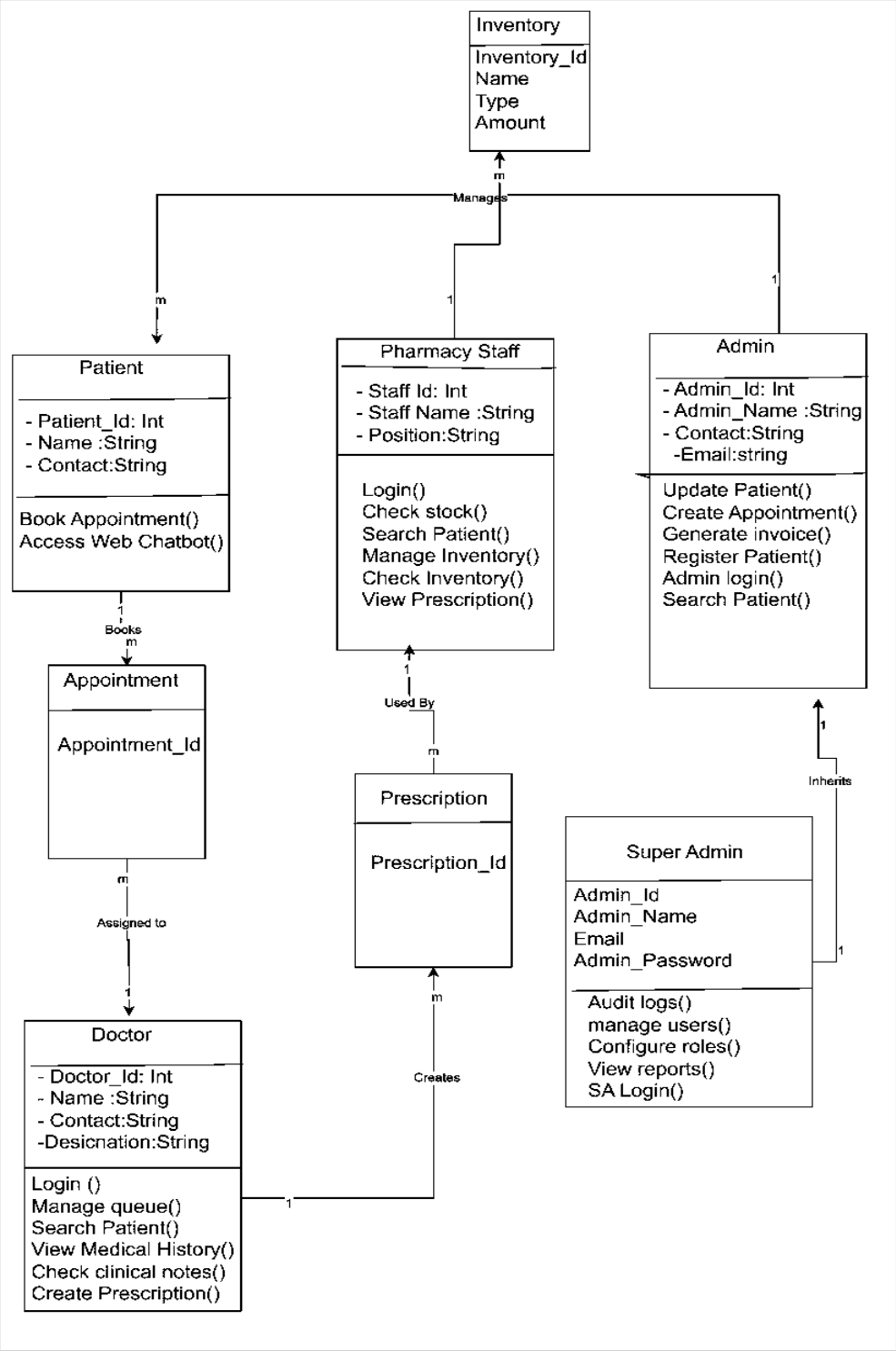


Figure 3-1 Class Diagram

3.1 Description of the Class Diagram

The system is centered around managing patients and medical services efficiently:

- Patient

  Individuals receiving medical services are represented by the Patient entity. Appointments may be booked by patients, and access to basic system features such as chatbot support is provided.

- Appointment

  Scheduling between patients and doctors is handled by the Appointment entity. Multiple appointments may be booked by a patient, and each appointment is assigned to a doctor.

- Doctor

  The diagnosis of patients is performed by a single doctor in the system. Patient queues are managed, prescriptions are created, and medical history and clinical notes are accessed by the doctor.

- Prescription

  Prescriptions are created by doctors after consultation. They contain treatment or medicine details and are later used by pharmacy staff.

- Pharmacy Staff

  Medicine-related operations such as stock checking, prescription viewing, and inventory updates are managed by pharmacy staff. Medication is dispensed based on prescriptions.

- Inventory

  Details of medicines such as name, type, and quantity are stored in the Inventory entity. It is managed by both Admin and Pharmacy Staff to ensure drug availability.

- Admin

  Administrative tasks such as patient registration, appointment creation, invoice generation, and patient detail updates are handled by the admin role.

- Super Admin

  A higher-level role is defined as Super Admin, through which advanced privileges such as user management, role configuration, report viewing, and audit log maintenance are provided.

3.2 Relationships Overview

- A Patient may have multiple Appointments (1 → many).

- Each Appointment is assigned to a single doctor in the system.
- A Doctor creates multiple Prescriptions.
- Prescriptions are used by Pharmacy Staff to dispense medicine.
- Inventory is managed by both Admin and Pharmacy Staff.
- Super Admin extends the admin class with additional control features.

CHAPTER 4: SEQUENCE DIAGRAM

The sequence diagrams are used to illustrate how various users, including Admin, Doctor, Pharmacy Staff, and Super Admin, interact with the system to perform different operations. The step-by-step flow of messages between actors, the system, and databases is presented in a time-ordered manner.

Key functionalities such as patient registration, appointment handling, prescription management, inventory control, billing, user management, and audit logging are covered in these diagrams. The way requests are sent, processed, and responses are returned is clearly depicted, ensuring proper coordination between system components.

Overall, the dynamic behavior of the system is demonstrated through sequence diagrams, and the interaction between different modules to support efficient clinic management is effectively represented.

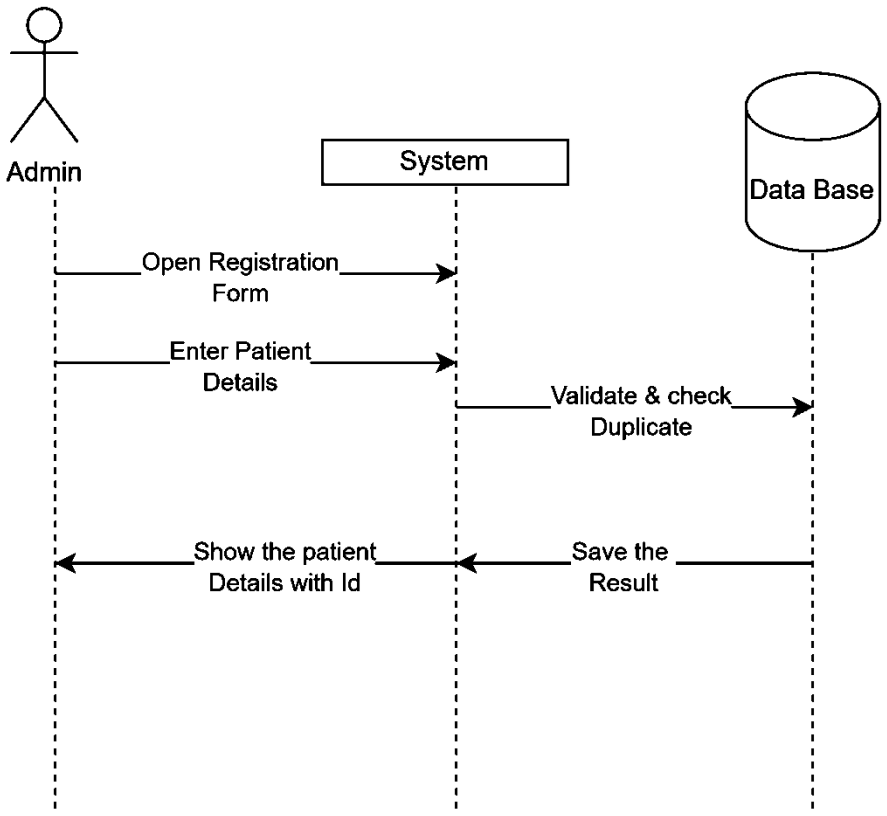


Figure 4-1 Sequence Diagram for Login

4.1 Sequence Diagram for Patient

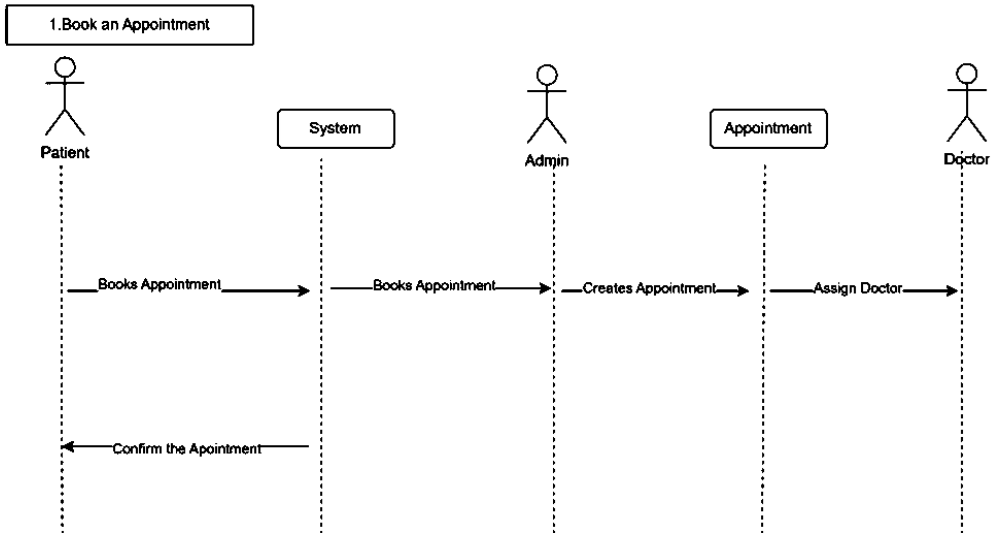


Figure 4-2 Sequence Diagram for Book Appointment

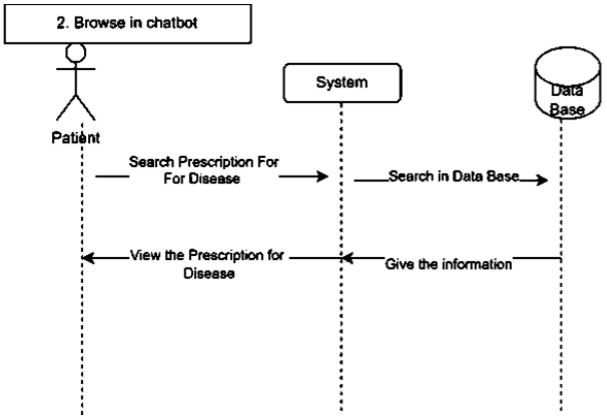


Figure 4-3 Sequence Diagram for Chat with Chatbot

4.2 Sequence Diagram for Doctor

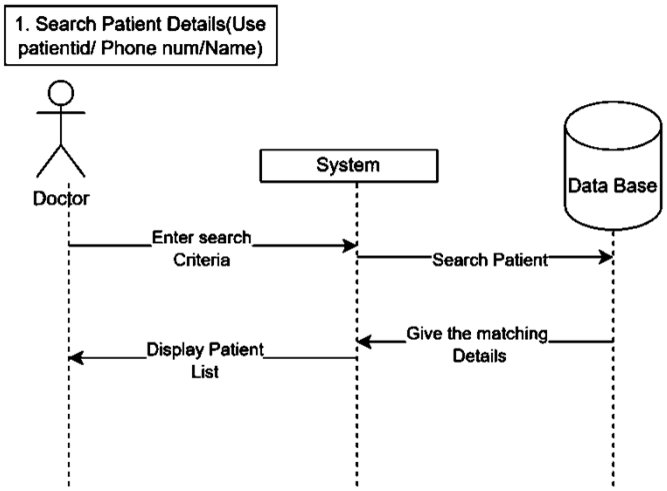


Figure 4-4 Sequence Diagram for Search Patient

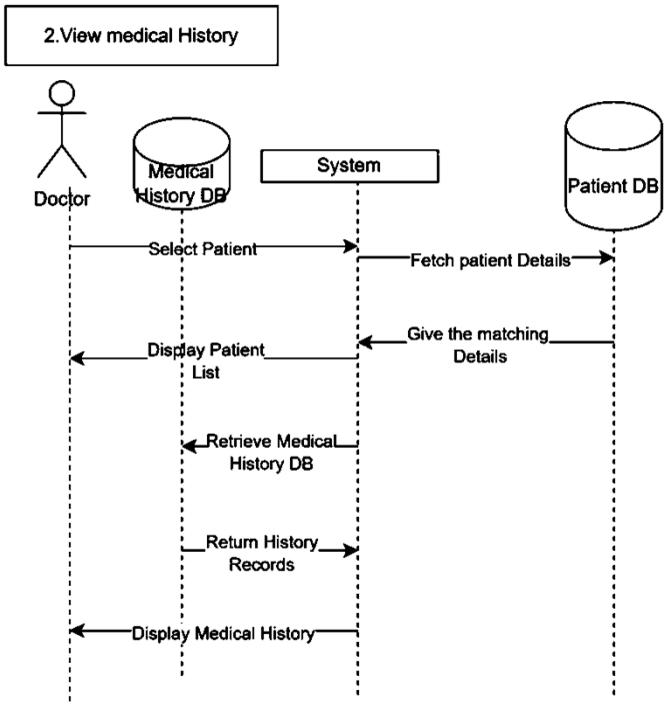


Figure 4-5 Sequence Diagram for View Medical History

4.3 Sequence Diagram for Pharmacy Staff

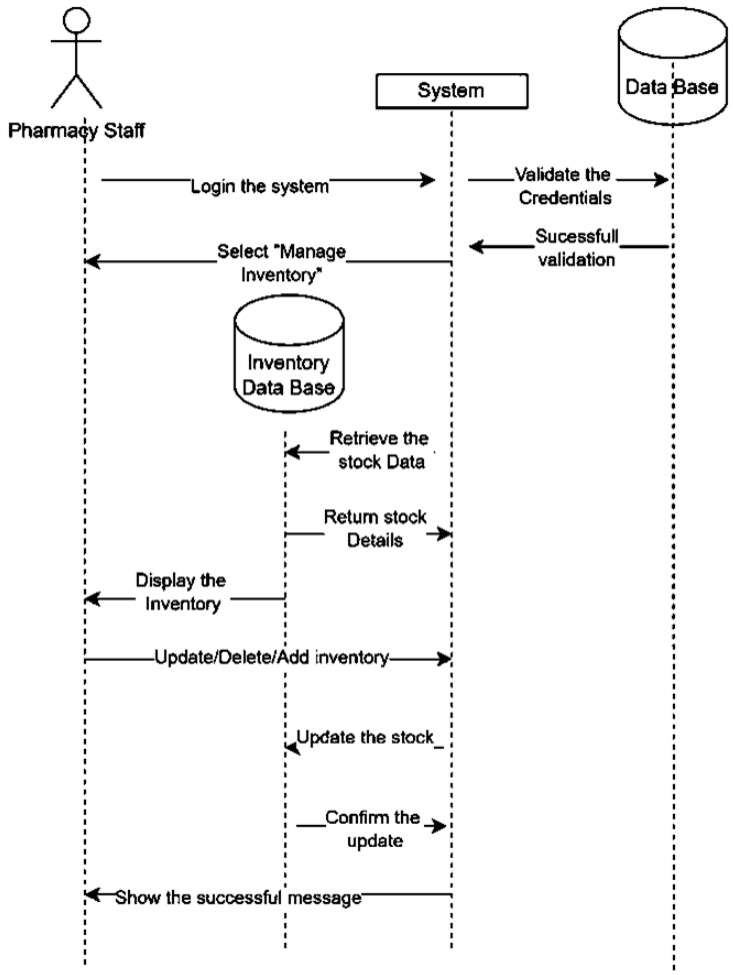


Figure 4-6 Sequence Diagram for Manage Inventory

4.4 Sequence Diagram for Admin

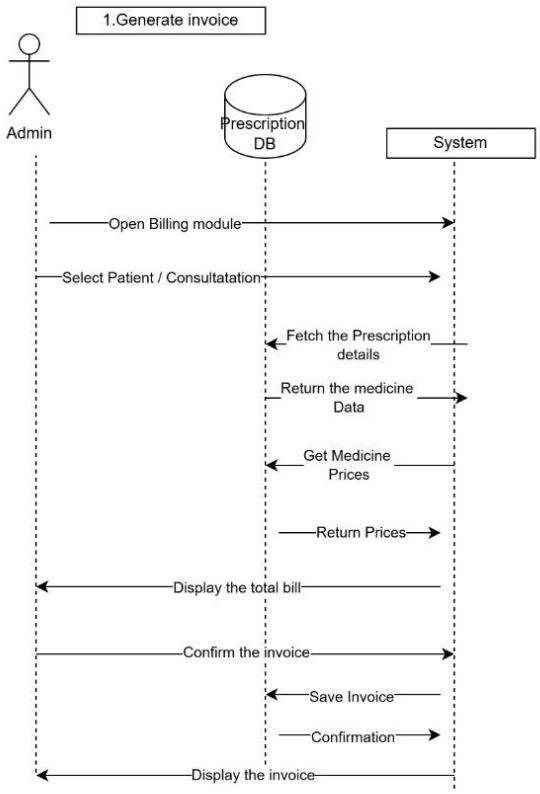


Figure 4-7 Sequence Diagram for Generate Invoice

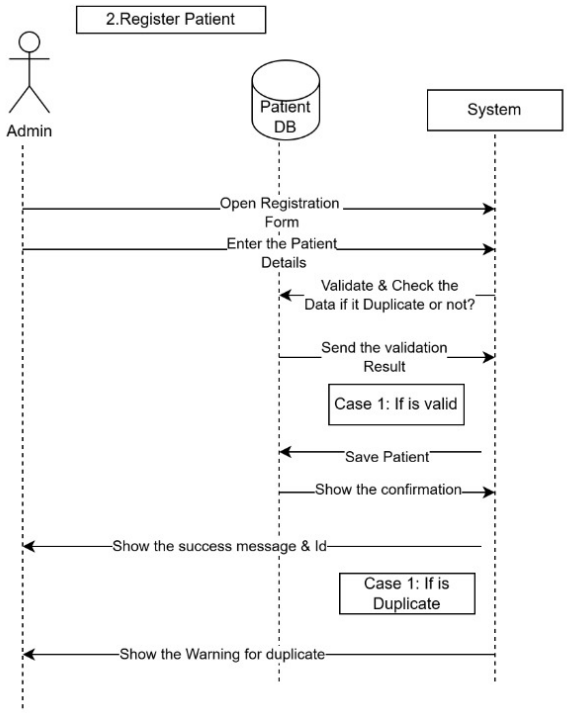


Figure 4-8 Sequence Diagram for Register Patient

4.5 Sequence Diagram for Super Admin

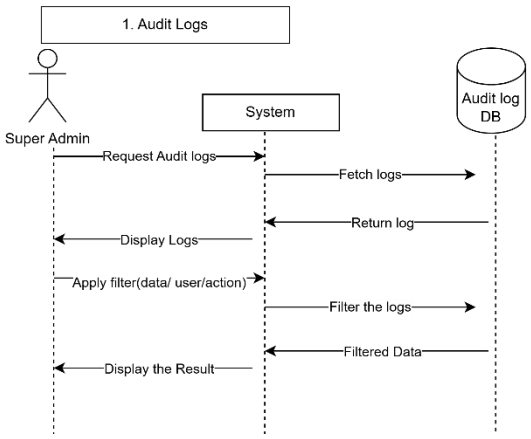


Figure 4-9 Sequence Diagram for Audit logs View

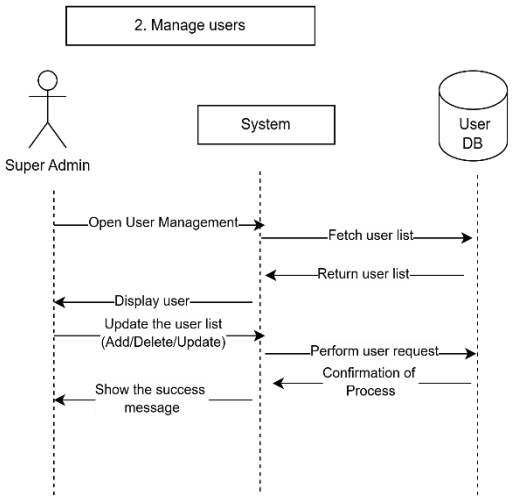


Figure 4-10 Sequence Diagram for Manage Users

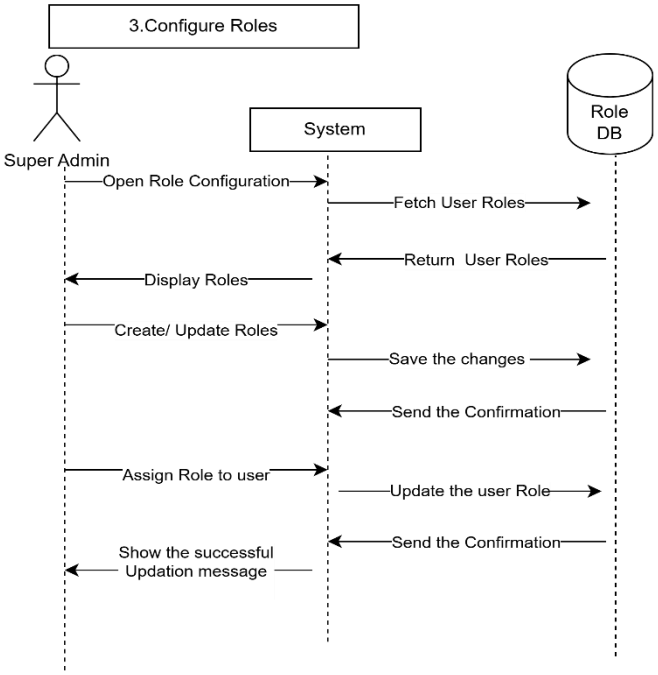


Figure 4-11 Sequence Diagram for Configure Roles

CHAPTER 5: UI/UX DESIGN

5.1 Wireframes and Screen Mock-ups

The user interface of the OneCare Clinic Management System is designed using a user-centered approach to ensure usability, efficiency, and clarity across different user roles.

Wireframes and high-fidelity mock-ups have been created using Figma to visualize the layout, structure, and interaction of key system components. These designs provide a clear representation of user interfaces before implementation.



Figure 5-1 Login Screen Wireframe

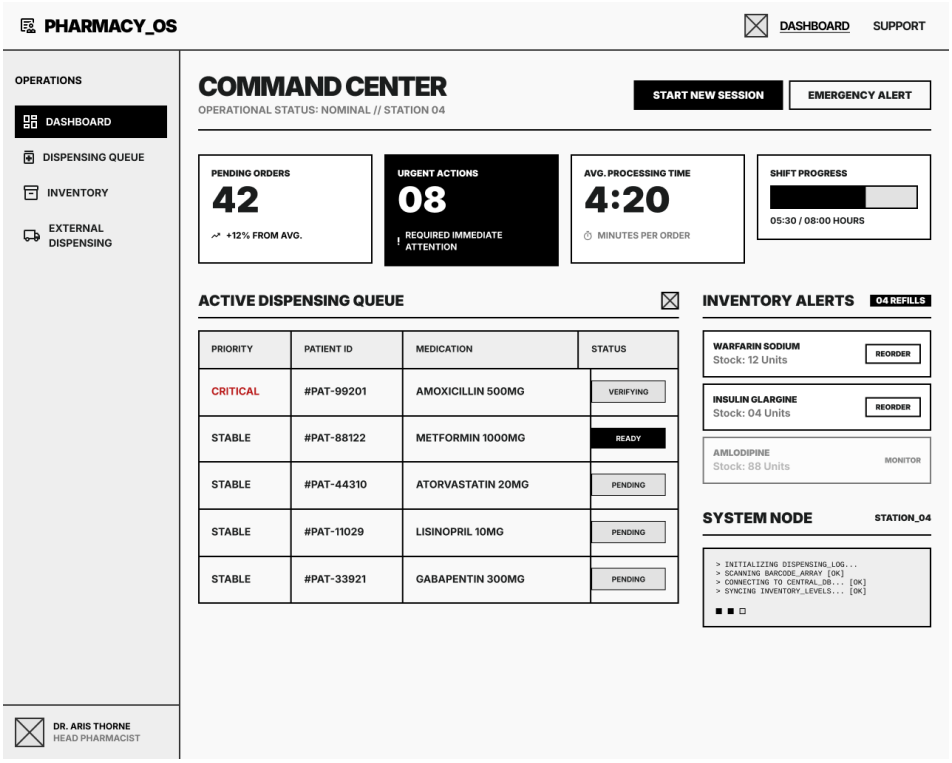


Figure 5-2 Pharmacist Dashboard

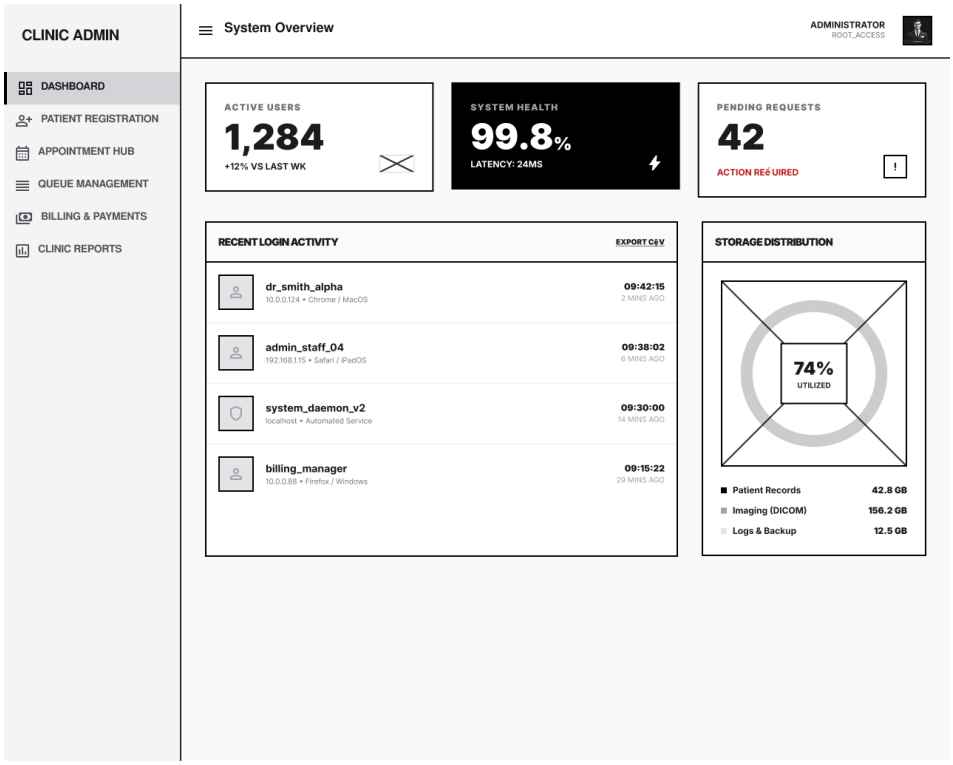


Figure 5-3 Admin's Dashboard

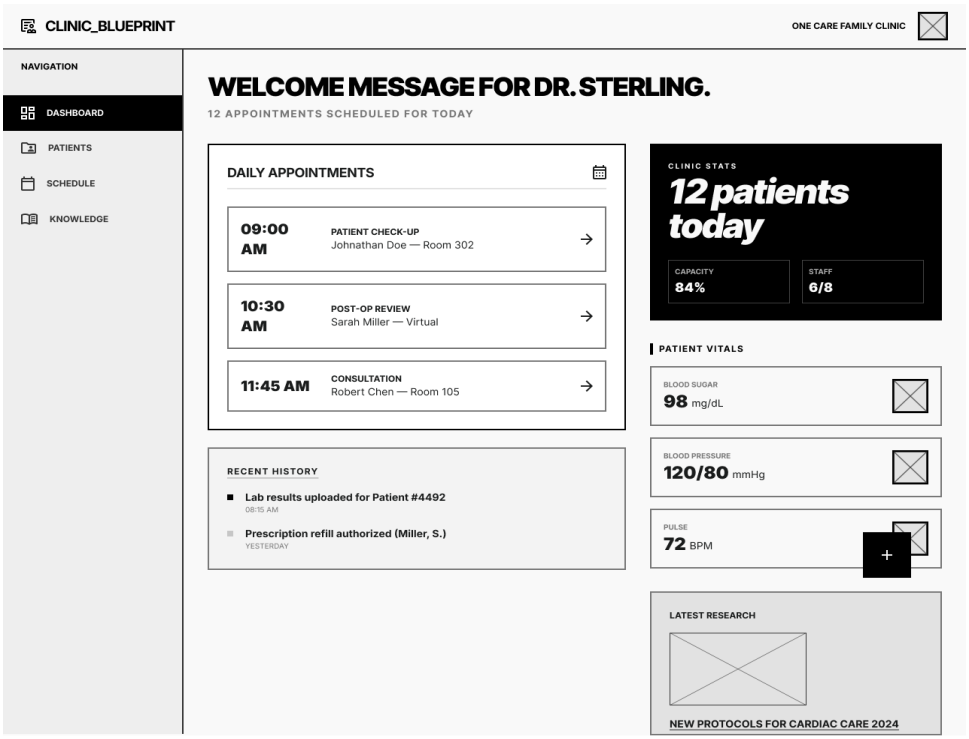


Figure 5-5 Doctor's Dashboard

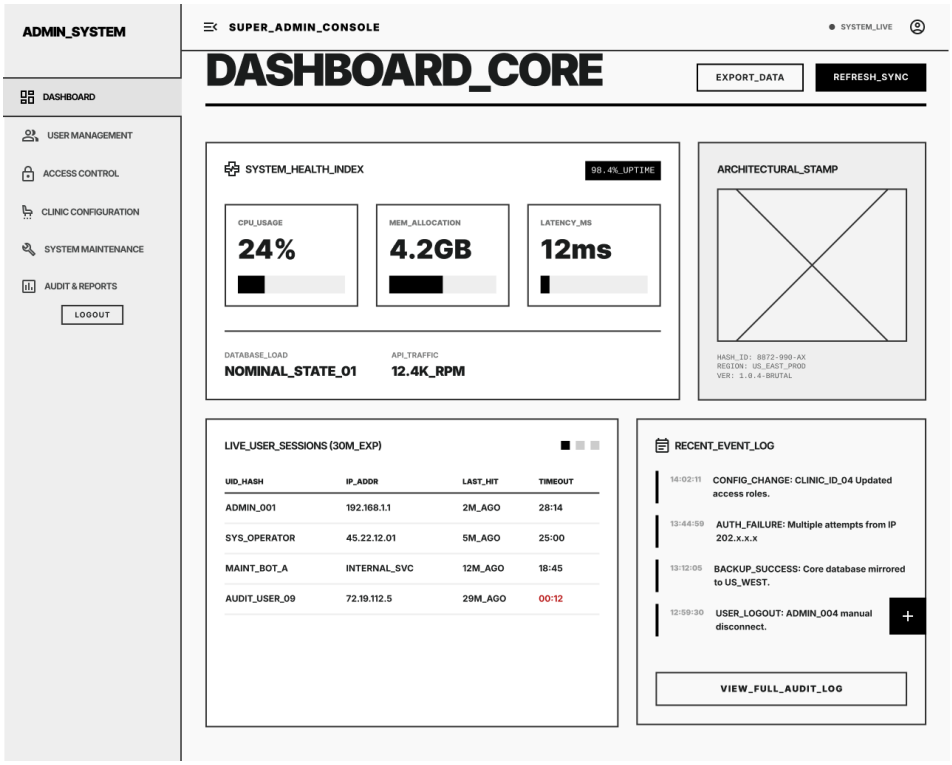


Figure 5-4 Super Admin's Dashboard

5.1.1 Key Screens

1. Login Interface

- Provides secure authentication for system users
- Includes input validation and error feedback
- Supports role-based access redirection after login

2. Role-Based Dashboard

- Displays summary information relevant to each role (Admin, Doctor, Pharmacist)
- Provides quick navigation to frequently used modules
- Includes alerts such as upcoming appointments and low inventory

3. Patient Management Interface

- Allows administrators to register and update patient information
- Displays patient records in a structured and searchable format

4. Appointment Management Interface

- Enables appointment scheduling, rescheduling, and cancellation
- Includes calendar-based and list-based views

5. Doctor Consultation Interface

- Allows doctors to view patient history and record clinical notes
- Supports prescription creation and submission

6. Pharmacy and Inventory Interface

- Displays prescriptions for dispensing
- Allows pharmacists to manage medicine inventory and stock levels

Each interface is designed with consistency in layout, typography, and component usage to ensure a seamless user experience.

5.2 User Flow and Navigation Design

The system navigation is designed to ensure smooth and efficient task completion with minimal user effort.

5.2.1 Primary User Flow

User → Login → Role-Based Dashboard → Select Module → Perform Action → Logout

Detailed Interaction Flow

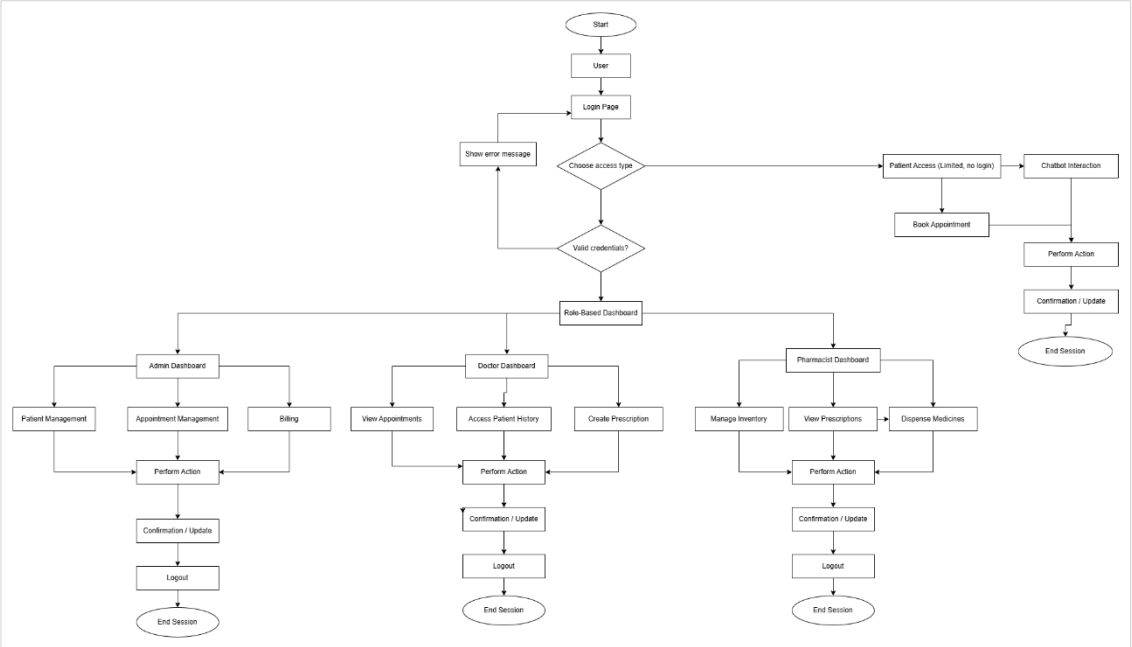


Figure 5-6 User flow Diagram

Navigation Structure

- Top navigation bar for global actions (profile, logout)
- Sidebar menu for module-based navigation
- Breadcrumbs for tracking user location within the system

This structure ensures:

- Reduced navigation complexity
- Faster task completion
- Clear system feedback

5.3 Usability Considerations

The user interface of the system has been designed with a strong focus on usability to ensure efficient and error-free interaction. Several usability principles have been considered during the design process:

- **Consistency:** Uniform layouts, colors, and components are maintained across all interfaces to reduce the learning curve for users.
- **Simplicity:** Interfaces are designed to be clean and uncluttered, allowing users to complete tasks with minimal effort.

- **Feedback Mechanisms:** Immediate system feedback is provided through alerts, validation messages, and confirmations to inform users about the outcome of their actions.
- **Error Prevention and Recovery:** Input validation and guided workflows are implemented to prevent errors, while clear error messages assist users in correcting mistakes.
- **Efficiency:** Frequently used features are easily accessible through dashboards and navigation menus, reducing the time required to perform tasks.

These usability considerations ensure that the system is intuitive, user-friendly, and suitable for users with varying levels of technical expertise, including administrative staff, medical professionals, and patients.

5.4 Accessibility Considerations (WCAG 2.1 Compliance)

The system is designed to comply with Web Content Accessibility Guidelines (WCAG) 2.1 to ensure usability for all users, including those with disabilities.

1. Visual Accessibility
    - Text and background color contrast ratio maintained at a minimum of 4.5:1
    - Avoidance of color-only indicators for important actions or errors
2. Keyboard Accessibility
    - All interactive elements (forms, buttons, navigation) are accessible via keyboard
    - Logical tab order is maintained across all screens
3. Screen Reader Support
    - Form inputs include descriptive labels
    - Buttons and interactive components have meaningful names
    - ARIA attributes are used where necessary
4. Error Handling and Feedback
    - Clear and descriptive error messages are displayed
    - Errors are not indicated by color alone; text explanations are provided
    - Inline validation helps users correct issues before submission
5. Responsive Design
    - Interfaces are optimized for desktops, tablets, and mobile devices
    - Layouts adapt dynamically to different screen sizes

The accessibility features implemented in the system are aligned with internationally recognized standards to ensure inclusivity and usability for all users. The design follows the guidelines specified in the Web Content Accessibility Guidelines (WCAG) 2.1, which define best practices for improving accessibility across web-based applications. These guidelines ensure that the system is perceivable, operable, understandable, and robust for users with diverse abilities.

CHAPTER 6: SECURITY DESIGN

6.1 Authentication Mechanism

A credential-based authentication mechanism is implemented using JSON Web Tokens (JWT) combined with username and password authentication. Secure identity verification, session management, and access control are enforced throughout the system.

6.1.1 Credential-Based Authentication

Authentication is performed using a username and password. Upon successful authentication, a JWT token is issued and used for subsequent requests.

6.1.2 Password Policy

The system enforces a strong password policy to enhance security.

Table 6-1 Password Policy

Rule	**Specification**
Minimum length	12 characters
Complexity	Uppercase, lowercase, numeric, and special character required
Hashing algorithm	Argon2id
First-login reset	Mandatory password change after reset
Password history	Last 5 passwords cannot be reused
Password expiry	Every 90 days

6.1.3 Password Storage

Passwords are securely stored using the Argon2id hashing algorithm with the following configuration:

- Algorithm: Argon2id
- Salt Length: 16 bytes (random per user)
- Time Cost: 2 iterations
- Memory Cost: 65,536 KB
- Parallelism: 1
- Hash Format: $argon2id$v=19$m=65536,t=2,p=1$<salt>$<hash>

6.1.4 Login Flow

The authentication process follows a secure sequence:

1. Credentials are submitted via POST /api/auth/login.
2. Input validation is performed (null checks, length validation).

3. The user record is retrieved from the database.
4. The password is verified using Argon2id hashing.

Successful Authentication

- A JWT token is generated.
- Login activity is recorded in the Audit Log (timestamp and client IP).
- The token is returned to the client.

Failed Authentication

- Failed attempt counter is incremented.
- If failed attempts ≥ 5:
    - The account is locked for 15 minutes.
    - Lockout event is recorded in the Audit Log.
    - A lockout response is returned.
- Otherwise:
    - An "Invalid username or password" message is returned.

6.1.5 Authentication Flow Diagram

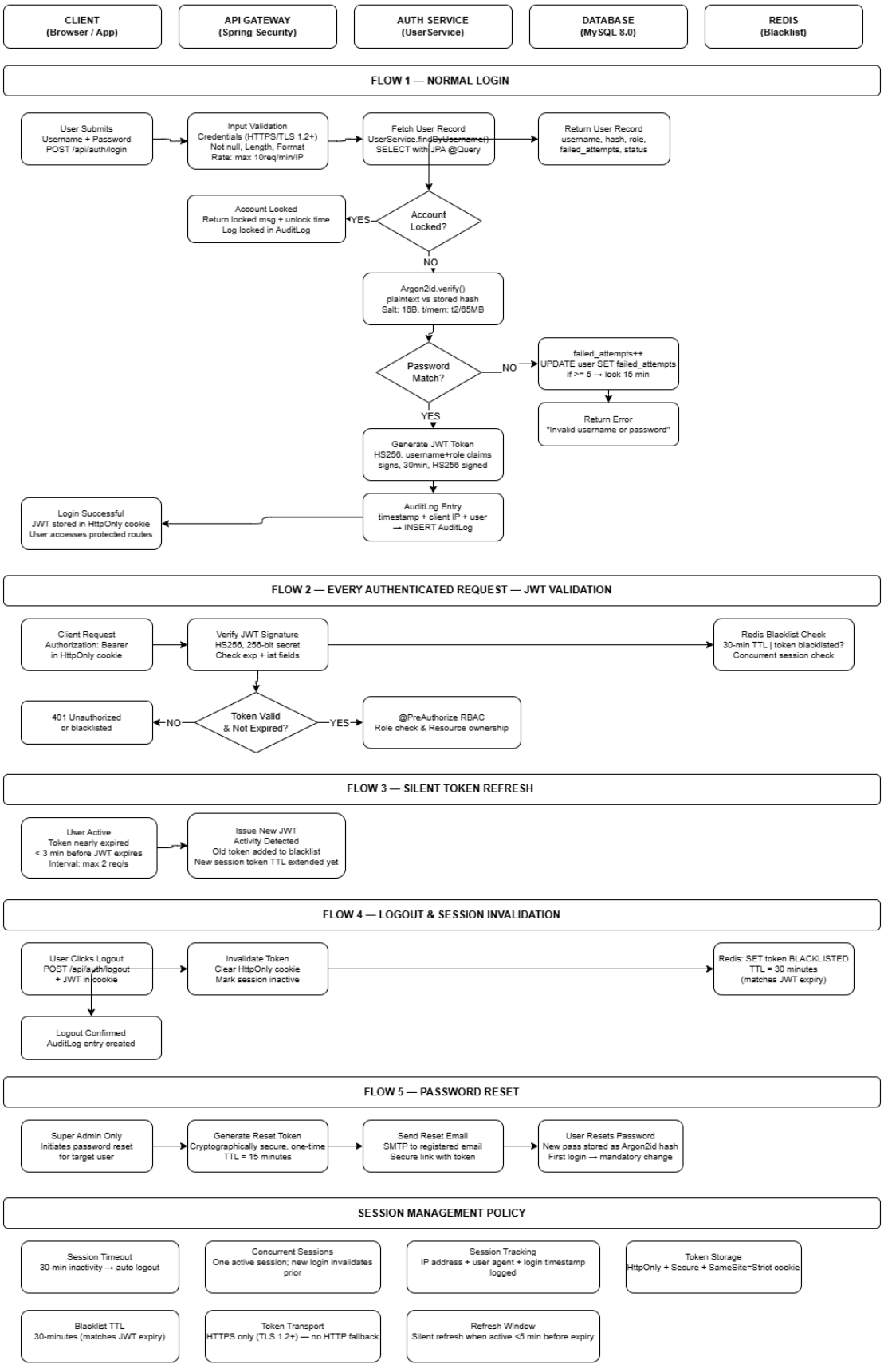


Figure 6-1 Authentication Flow Diagram

6.1.6 JWT Token Structure

JWT tokens are issued as Bearer tokens and transmitted via the HTTP Authorization header. Tokens are stored securely in cookies.

Token Components

Header:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload:

```
{
  "sub": "user_123",
  "username": "doctor_name",
  "role": "DOCTOR",
  "email": "doctor@clinic.com",
  "iat": 1712282400,
  "exp": 1712284200,
  "iss": "onecare-clinic",
  "aud": "onecare-app"
}
```

Signature:

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  SECRET_KEY
)
```

6.1.7 Token Security Properties

- Signing: HMAC-SHA256 (256-bit key)
- Key storage: Secure environment variables
- Transport: HTTPS (TLS 1.2+)
- Storage: httpOnly, Secure, SameSite=Strict cookie
- Token refresh: Automatic refresh before expiry (5 minutes prior)
- Token revocation: Redis-based blacklist (30-minute TTL)

6.1.8 Session Management

The system enforces strict session control policies:

- Session timeout: 30 minutes of inactivity

- Concurrent sessions: Single active session per user
- Session tracking: IP address, user agent, login timestamp
- Logout behavior: Token blacklisted and session invalidated

6.1.9 Password Reset Flow

The password reset process is controlled and secure:

1. Reset is initiated by a Super Administrator.
2. A time-limited reset token is generated.
3. A reset link is sent via email (SMTP).
4. The link remains valid for 30 minutes and is single use.
5. The user is required to change the password upon first login.
6. The reset event is recorded in the Audit Log.

6.1.10 Authentication Mechanisms Summary

- Credential storage: Argon2id hashing (16-byte salt, 65536 KB memory)
- Token type: JWT (Bearer, HS256)
- Token expiry: 30-minute session timeout
- Token transport: HTTPS + httpOnly Secure cookie
- Account lockout: 15 minutes after 5 failed attempts
- Password reset: Email-based, 30-minute expiry, one-time use
- Rate limiting: Maximum 10 authentication requests per minute per IP

6.2 Authorization Levels

6.2.1 Role Definitions

Five system roles are defined with strictly controlled permissions to enforce separation of duties.

Table 6-2 Role Definitions

Role	Key Responsibilities
Super Administrator	User management, role configuration, audit log access, system backup and restore, chatbot knowledge base management
Administrator	Patient registration and updates (non-clinical), appointment and queue management, billing and payments
Doctor	Appointment viewing, medical history access, clinical note recording, prescription creation
Pharmacist	Prescription dispensing, inventory management, and external dispensing operations

Patient	Appointment booking (without authentication) and chatbot interaction

To enforce proper separation of responsibilities:

- Doctors are restricted from modifying pharmacy inventory.
- Pharmacists are restricted from modifying patient medical records.
- Administrators are limited to non-clinical patient data only.

These restrictions are enforced at multiple layers within the system.

6.2.2 Permission Matrix

Table 6-3 Permission Matrix

Module	Action	Super Admin	Admin	Doctor	Pharmacist	Patient
User Management	Create User	✓	✗	✗	✗	✗
User Management	Edit/Deactivate User	✓	✗	✗	✗	✗
User Management	Reset Password	✓	✗	✗	✗	✗
Clinic Configuration	Configure Schedules	✓	✗	✗	✗	✗
Clinic Configuration	Manage Roles & Permissions	✓	✗	✗	✗	✗
Patient Management	Register Patient	✓	✓	✗	✗	✗
Patient Management	Edit Non-Clinical Information	✓	✓	✗	✗	✗
Patient Management	View Patient Profile	✓	✓	✓	✓*	✗
Appointment Management	Create Appointment	✓	✓	✗	✗	✓ (self)
Appointment Management	Reschedule/Cancel	✓	✓	✗	✗	✗

Appointment Management	View Appointments	✓	✓	✓ (own)	✗	✓ (own)
Queue Management	Manage Queue	✓	✓	✓	✗	✗
Doctor Module	Access Medical History	✓	✗	✓	✗	✗
Doctor Module	Record Clinical Notes	✗	✗	✓	✗	✗
Doctor Module	Create Prescription	✓	✗	✓	✗	✗
Pharmacy	View Prescriptions	✓	✗	✓	✓	✗
Pharmacy	Dispense Medicines	✓	✗	✗	✓	✗
Inventory	Manage Medicines	✓	✗	✗	✓	✗
Inventory	View Stock Levels	✓	✗	✓ (read-only)	✓	✗
External Dispensing	Mark External Dispense	✓	✗	✗	✓	✗
External Dispensing	Verify Patient Identity	✓	✗	✗	✓	✗
Billing	Generate Invoice	✓	✓	✗	✗	✗
Billing	Record Payment	✓	✓	✗	✗	✗
Reports	View Clinic Reports	✓	✓	✗	✗	✗
Reports	View Pharmacy Reports	✓	✗	✗	✓	✗
Chatbot	Manage Knowledge Base	✓	✗	✓	✗	✗

Audit	View Audit Logs	✓	✗	✗	✗	✗
System	Backup & restore	✓	✗	✗	✗	✗

* Pharmacists are limited to non-clinical patient information only.

6.2.3 RBAC Implementation

Authorization is enforced using a multi-layered approach to ensure secure access control.

- **Endpoint-Level Security:** Access control is enforced using framework-level annotations.
- **Service-Level Authorization:** Business rules are enforced within the service layer to prevent unauthorized operations.
- **Data-Level Security:** Access restrictions are enforced at the data layer using filtered queries.

6.3 Data Protection Approach

6.3.1 Encryption at Rest

Sensitive data is protected at rest using strong encryption mechanisms at both the database and application levels.

Table 6-4 Data Protection Approach

Data Category	**Encryption Method**	**Algorithm**	**Key Size**
Patient Records	Database-level encryption (InnoDB TDE)	AES-256	256-bit
Authentication Credentials	Secure hashing	Argon2id	—
Sensitive Fields (e.g., contact details, healthcare ID)	Column-level encryption	AES-256-CBC (PKCS5 padding)	256-bit
Prescription / Clinical Notes	Column-level encryption	AES-256-CBC	256-bit
Backup Files	File-level encryption	AES-256-GCM	256-bit

Audit Logs	Encrypted database storage	AES-256 (TDE)	256-bit
External Dispensing Records	Column-level encryption	AES-256-CBC	256-bit

Implementation Details

- Transparent Data Encryption (TDE) is enabled at the database level to secure stored data files.
- Sensitive fields are encrypted at the application layer using secure cryptographic libraries.
- Encryption keys are stored separately from application source code and managed through secure configuration mechanisms (e.g., environment variables or a secrets manager).
- A key rotation policy is enforced, where encryption keys are rotated periodically and securely archived to support long-term data access requirements.

6.3.2 Encryption in Transit

All data transmitted between clients and servers is secured using industry-standard transport encryption protocols.

- Protocol: HTTPS with TLS 1.2 or higher
- Cipher Suites: Strong Cipher suites (e.g., AES-256-GCM-based)
- Certificate: Valid SSL/TLS certificate issued by a trusted Certificate Authority
- HSTS: HTTP Strict Transport Security enforced
- Perfect Forward Secrecy: Enabled using ephemeral key exchange mechanisms

This ensures that data cannot be intercepted or modified during transmission.

6.3.3 Data Access Control

Access to data is restricted based on user roles and responsibilities to ensure confidentiality and integrity.

- **Patient medical records** are accessible only to authorized medical personnel.
- **Prescription data** is accessible to the issuing doctor and authorized pharmacy staff for dispensing purposes.
- **Non-clinical patient data** is accessible to administrative staff for operational use.
- **Financial and billing data** is restricted to authorized administrative roles.
- **Audit logs** are accessible only to highly privileged system roles.

- **External dispensing records** are accessible only to authorized pharmacy personnel.
- **Chatbot knowledge base content** is manageable only by authorized roles.
- **Sensitive data masking** is applied in reports, where partial values (e.g., contact numbers, identification numbers) are displayed instead of full data.

6.4 Input Validation and Sanitization Strategy

6.4.1 Input Validation Strategy

Input validation is enforced across multiple layers to ensure data integrity and prevent invalid or malicious input.

Layer 1 - Client-Side Validation

Validation is performed at the user interface level to provide immediate feedback and reduce invalid submissions:

- Required field checks are enforced.
- Email format validation is applied.
- Phone number format validation follows the Sri Lankan format (+94-XX-XXX-XXXX).
- Date validation ensures that appointment dates are not set in the past.
- Numeric constraints are enforced (e.g., stock quantity ≥ 0, price > 0).
- Inline validation messages are displayed prior to form submission.

Layer 2 - Server-Side Validation

Server-side validation is implemented using Bean Validation (JSR-380) to ensure that all incoming data is validated regardless of client-side enforcement.

Layer 3 - Business Logic Validation

Additional validation rules are enforced within the service layer:

- Doctor availability is verified before confirming appointments.
- Appointment slot conflicts are prevented to avoid double-booking.
- Prescriptions are validated against available and approved medicines.
- Inventory updates are restricted to prevent negative stock levels.
- External dispensing quantities are validated against remaining prescription limits.
- Patient identity verification is required before dispensing medications externally.

6.4.2 Sanitization Strategy

Sanitization mechanisms are applied to prevent injection attacks and ensure safe data handling.

Cross-Site Scripting (XSS) Prevention

- User-supplied inputs are sanitized before storage and rendering.
- Special characters such as <, >, &, ", and ' are escaped.
- Frontend rendering frameworks automatically escape output values, providing an additional layer of protection.

SQL Injection Prevention

- All database operations are executed using parameterized queries.
- Object-relational mapping frameworks are used to eliminate direct SQL string construction.

6.4.3 CSRF Protection

Cross-Site Request Forgery protection is implemented to secure state-changing operations.

- CSRF tokens are generated per session and included in all state-changing requests (e.g., POST, PUT, DELETE).
- Built-in security framework protections are enabled for all secured endpoints.
- Public endpoints are excluded from CSRF enforcement but are protected through alternative mechanisms such as rate limiting.

6.4.4 Rate Limiting

Rate limiting is applied to prevent abuse, brute-force attacks, and denial-of-service scenarios.

Table 6-5 Rate Limiting Scenarios

Endpoint Category	Limit	Enforcement Mechanism
Authentication endpoints	10 requests per minute per IP	Security filters with distributed counters
Public booking endpoints	20 requests per minute per IP	API gateway rate limiting
General API endpoints	100 requests per minute per user	API gateway rate limiting

Rate limiting policies are enforced dynamically and monitored to ensure system stability and availability.

6.5 Encryption Standards for data at rest and in transit

6.5.1 Encryption Standards

This section provides a consolidated overview of the encryption mechanisms implemented within the system.

Table 6-6 Encryption Standards

Standard	**Usage**	**Algorithm**	**Key Length**
AES-256-CBC	Column-level encryption (patient data, prescriptions)	AES	256-bit
AES-256-GCM	Backup file encryption	AES	256-bit
TLS 1.2+	Client–server communication	TLS with secure key exchange	—
Argon2id	Password hashing	Argon2id	128-bit output
HS256	JWT token signing	HMAC-SHA256	256-bit
RSA-2048	SSL/TLS certificates	RSA	2048-bit

6.5.2 Security Architecture Diagram

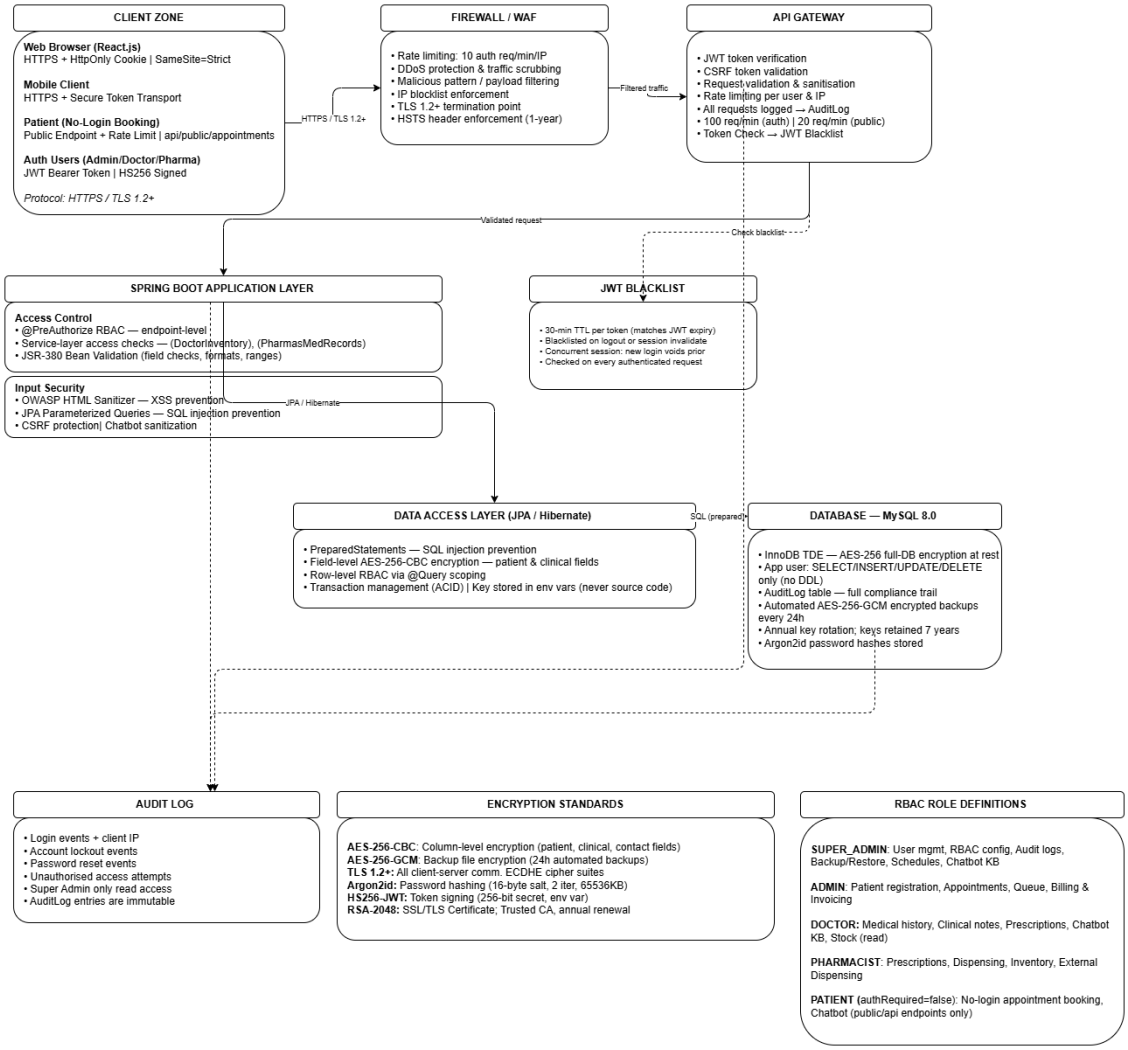


Figure 6-2 Security Architecture

6.5.3 Security Testing and Vulnerability Management

Security testing and vulnerability management practices are implemented to ensure continuous monitoring, detection, and mitigation of security risks.

Table 6-7 Security Testing

Activity	**Tool / Method**	**Frequency**
Dependency vulnerability scanning	Automated dependency analysis tools	Every build
Static code analysis	Code quality and security analysis tools	Every build

Dynamic security testing	Automated application security scanning	Before production releases
Penetration testing	External or internal security assessment	Periodically
Authentication and access monitoring	Centralized logging and alerting systems	Continuous
Security patch management	Regular updates and vulnerability fixes	As required
Incident response	Defined response procedures (detection, containment, recovery)	Triggered upon incidents

CHAPTER 7: DEPLOYMENT DESIGN

7.1 Hosting Plan

The OneCare Clinic Management System shall be deployed using containerized architecture to ensure portability, scalability, and ease of management. The system shall initially be hosted in a local development environment and can be extended to a cloud-based infrastructure (AWS) for production deployment.

7.2 Hardware Requirements

The minimum hardware requirements for system deployment are defined as follows:

- Processor: Intel i5 or higher
- RAM: Minimum 8 GB
- Storage: Minimum 256 GB SSD
- Network: Stable internet connection

7.3 Network Diagram

The system architecture shall include client devices communicating with the backend services through a secure network. The backend services shall interact with the database and external services such as SMS and email gateways.

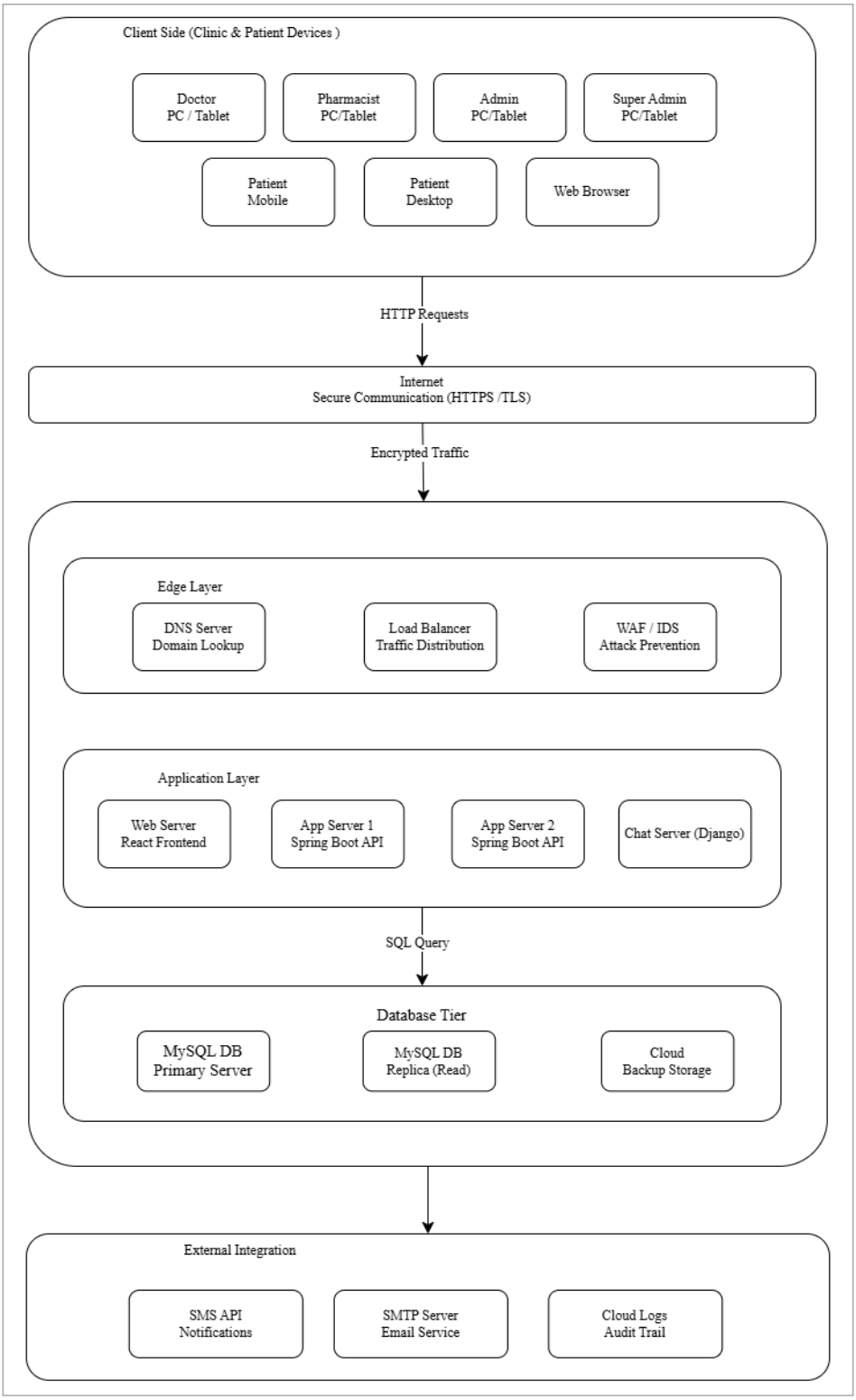


Figure 7-1 Network Diagram

7.4 Environment specification

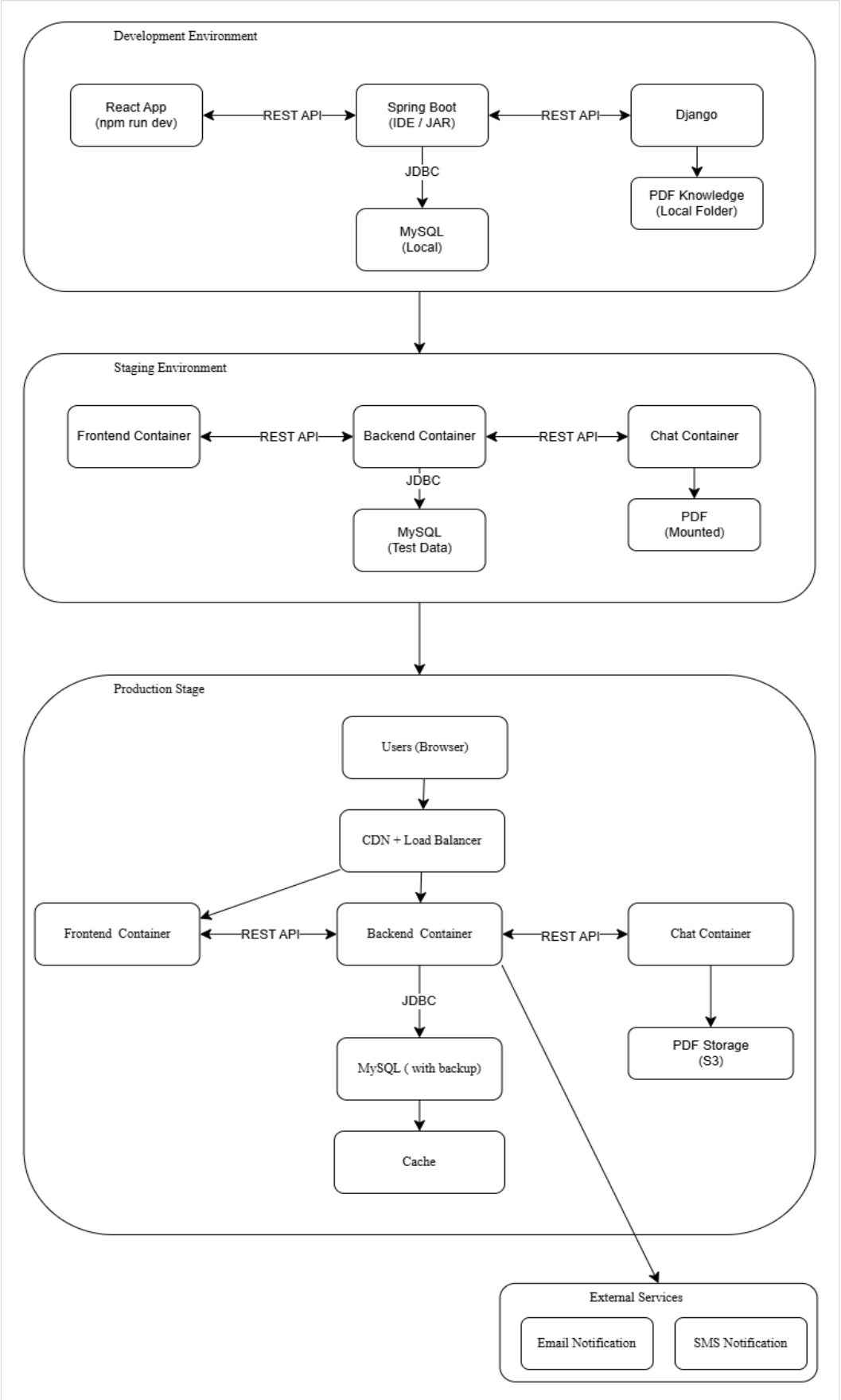


Figure 7-2 Environment Specification

7.4.1 Development Environment

The development environment shall be used by developers for coding, debugging, and initial testing. The system shall run locally using Docker Compose, enabling all services (frontend, backend, and database) to be executed in isolated containers.

7.4.2 Staging Environment

The staging environment shall replicate the production environment as closely as possible and shall be used for quality assurance (QA), integration testing, and user acceptance testing (UAT). Test data shall be used instead of real patient data to ensure data privacy.

7.4.3 Production Environment

The production environment shall represent the live system used in real-world clinic operations. It shall be designed to ensure high availability, security, and reliability.

The following operational targets and measures shall be enforced:

- System availability shall be maintained at **≥ 99%** (NFR-R-001)
- Disaster recovery shall be completed within **≤ 4 hours (RTO)** (NFR-R-005)
- Automated daily backups shall be performed with geo-redundancy
- Real-time monitoring and alerting mechanisms shall be implemented
- Centralized logging shall be maintained with a retention period of 90 days
- Role-Based Access Control (RBAC) shall be enforced across all modules
- Audit logging shall be implemented for compliance and traceability

7.5 CI/CD Pipeline

A Continuous Integration and Continuous Deployment (CI/CD) pipeline shall be implemented to automate the build, testing, and deployment processes.

The pipeline shall include the following stages:

1. **Source Control**

   Code shall be managed using Git-based repositories (GitHub).

2. **Build Stage**

   The application shall be built automatically upon code commits.

3. **Testing Stage**

   Automated unit and integration tests shall be executed to ensure code quality.

4. **Containerization**

   Docker images shall be created for the frontend and backend services.

5. **Deployment**

   The application shall be deployed to the target environment (staging or production).

6. **Monitoring**

   Deployed services shall be monitored for performance, errors, and uptime.

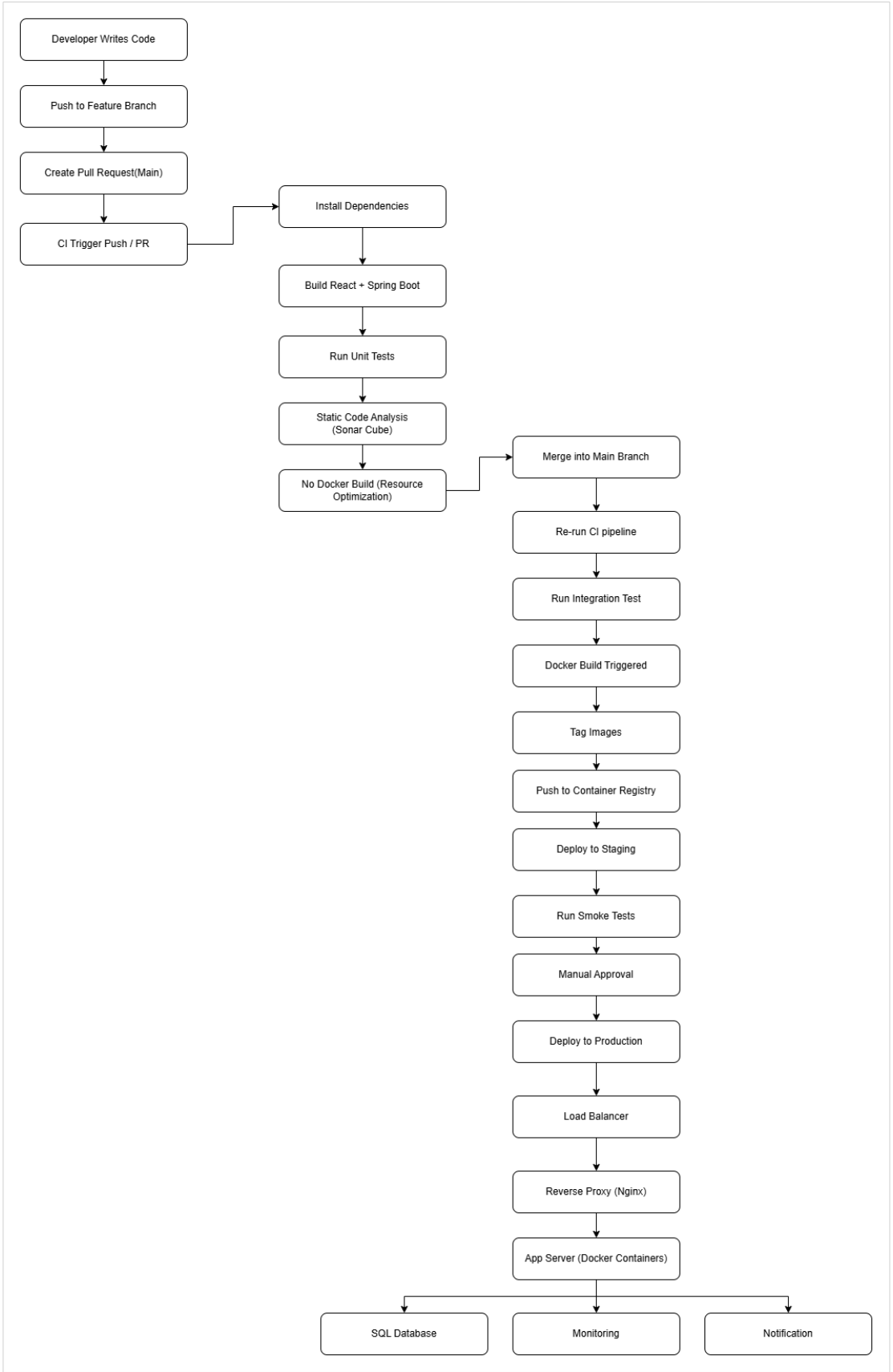


Figure 7-3 CI/CD pipeline architecture

REFERENCES

[1] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” World Wide Web Consortium, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

APPENDICES

All the diagrams can be accessed using this link: Google Drive